ARCAdevint

Reference manual

By

ARCA Technologies s.r.l.
Via Statale, 17
10012 - Bollengo (TO)

Tel. +39-0125.235.611
Fax. +39-0125.235.623

Website: www.arca.com
E-mail: techsupp.it@arca.com

Contents

1 Overview 4
1.1 Revisions 4
2 Product Description 6
2.1 Architecture 6
2.2 Technologies 6
2.3 Compatibility 7
2.4 Supported devices 7
2.5 Integration 8
3 Functions 9
3.1 ARCAInitialize 9
3.2 ARCAConnect 10
3.3 ARCADisconnect 16
3.4 ARCADeviceStatus 17
3.5 ARCAOpen 26
3.6 ARCAClose 28
3.7 ARCAgetCashData 29
3.8 ARCADepositStart 45
3.9 ARCADeposit 47
3.10 ARCADepositEnd 56
3.11 ARCACount 61
3.12 ARCADispense 66
3.13 ARCAgetDeviceDateTime 70
3.14 ARCAsetDeviceDateTime 72
3.15 ARCAexeCassetteEmpty 74
3.16 ARCAgetDeviceVersion 77
3.17 ARCAgetDeviceType 83
3.18 ARCAexeDeviceReset 87
3.19 ARCAgetReaderCurrency 89
3.20 ARCAgetReaderBanks 91
3.21 ARCAgetReaderVersion 93

3.22 ARCAgetCassetteSettings 95
3.23 ARCAgetReaderStatistics 104
3.24 ARCAsetUpperTransportLock 107
3.25 ARCAsetCassetteSettings 109
3.26 ARCAsetConfigReader 111
3.27 ARCAgetMacAddress 113
3.28 ARCAgetReaderThreshold 115
3.29 ARCAgetDeviceSettings 117
3.30 ARCAsetAlarm 121
3.31 ARCAgetDelays 123
3.32 ARCAgetAudioSettings 127
3.33 ARCAexeAlarm 129
3.34 ARCAgetReaderIdentifier 131
3.35 ARCAInitAsyncEvent 134
3.36 ARCAsetDelays 136
3.37 ARCAexeTraceTransfer 138
3.38 ARCAexeCassetteInit 140
3.39 ARCAgetPocketOutput 142
3.40 ARCAsetPocketOutput 144
3.41 ARCAgetPocketInput 146
3.42 ARCAsetPocketInput 148
3.43 ARCAsetLogServiceRecord 150
3.44 ARCAgetModuleSerialNumber 152
3.45 ARCAgetCoinCashData 154
3.46 ARCACoinInventory 156
3.47 ARCACoinDispense 158
3.48 ARCAgetCoinConfiguration 160
3.49 ARCAgetCassettePhysicalPosition 162
~~3.50 ARCAgetBanknoteSerialNumber~~ 165
3.51 ARCAexeDeviceBell 167

1 Overview

Project Name	*ARCAdevint*
Project Owner	*Loggia Davide*

1.1 Revisions

Date	Editor	Description	Version
04/04/22	Loggia Davide		
07/04/22	Davide Loggia	*fix TraceFunction in case of NULL buffer ,* *Rename file from NOVUS-errror.json to status.json*	0.8.7.0
22/04/22	Davide Loggia	*Add usb Chip Version in ARCAgetDeviceVersion* *Add ARCAsetDelays* *Add ARCAexeTraceTransfer* *Add ARCAexeCassetteInit* *Add ARCAgetPocketOutput* *Add ARCAsetPocketOutput* *Add ARCAgetInputSetting* *Add ARCAsetLogServiceRecord* *Fix Compile outData in case of loss connection*	0.9.0.0
10/05/22	Davide Loggia	ARCAInitialize add reply code (5001-5006) ARCAgetDeviceType add “type” : "machineExtended"	0.9.1.0
17/05/22	Davide Loggia	Add ARCAgerModuleSerialNumber	0.9.2.0
30/05/22	Davide Loggia	Add ARCAgetCoinCashData, ARCACoinInventory, ARCACoinDispense, ARCAgetCoinConfiguration	0.9.3.0
17/06/22	Davide Loggia	ArcaDispense add 30 minute TimeOut add parameter digitalDistribution to ARCAgetDeviceVersion	0.9.4.0
15/07/22	Davide Loggia	Add ARCAgetCassettePhysicalPosition	0.9.5.0
22/07/22	Davide Loggia	ARCAexeTraceTransfer -> integrate ARCAdevlnk.dll (1.4.1.0) to download the trace	0.9.6.0
26/09/2022	Davide Loggia	Add osVersion to ARCAgetDeviceVersion Add ARCAgetBanknoteSerialNumber	0.9.7.0
05/10/2022	Davide Loggia	Add assetVersion to ARCAgetDeviceVersion	0.9.8.0
25/10/2022	Davide Loggia	*Add ARCAgetAudioSettings* *Add ARCAexeDeviceBell*	0.9.9.0
15/11/2022	Davide Loggia	*Internal Release*	0.9.10.0
21/11/2022	Davide Loggia	*Add internal code for certification*	0.9.11.0

06/12/2022	Davide Loggia	*Add* AR_NO_ARCA_DEVICE_CONNECTED(1793) on ARCAConnect in case of no ARCAdevice connected	0.9.12.0
21/12/2022	Davide loggia	*Removed obsolete functions*	0.9.13.0
23/01/2023	Davide Loggia	*ArcaCount mode parameter* managed (*singleEmis , multiEmis , standardCount , modeBlockCount)* *ARCAgetDeviceVersion add cdfname*	0.9.14.0
02/02/2023	Davide Loggia	*Correct timeout in Stop Deposit Continus feeding*	0.9.15.0
24/02/2023	Davide Loggia	*Add reply code 1750(*AR_INVALID_INDATA) *Fix rs323 connection*	0.9.16.0

2 Product Description

2.1 Architecture

Here a scheme with the DLLs architecture:

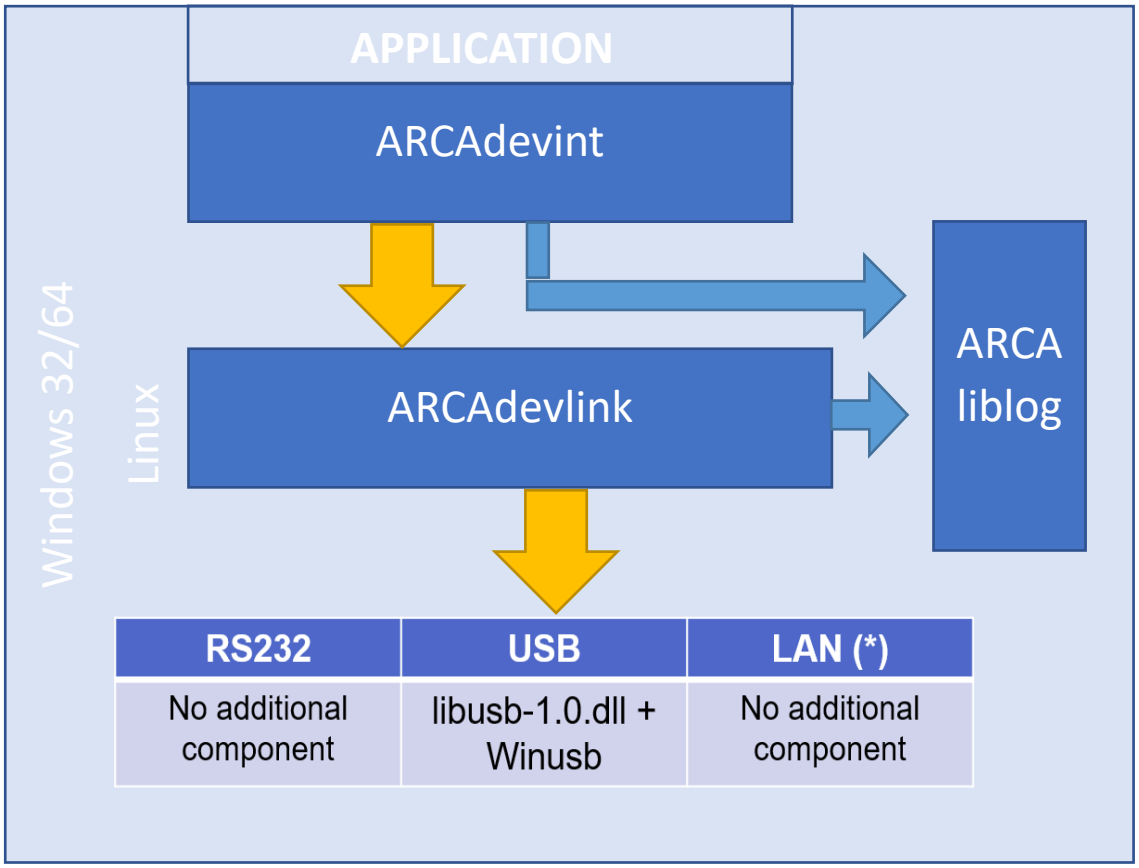
All functions are exported in C-Language compatible application programming interface (API).

The API is independent from the physical connection technology, whether it be TCP/IP, USB or RS232 used to connect the recycler to the host system.

To start an application program that uses this service it is necessary to install, in the application directory, the files listed below:

- ARCAdevint.dll
- ARCAdevlnk.dll
- ARCAliblog

Here a scheme with the DLLs architecture:



(*) TLS supported with OpenSSL (FUTURE ENHANCEMENT)

2.3 Compatibility

Windows 10 32/64bit , Windows11 32/64bit

Ubuntu Linux 32/64bit

2.4 Supported devices

CM18x, CM18Solo, Novus

2.5 Integration

All functions are exported in C-Language compatible application programming interface (API).

The API is independent from the physical connection technology, whether it be TCP/IP, USB or RS232 used to connect the recycler to the host system.

To start an application program that uses this service it is necessary to install, in the application directory, the files listed below:

- ARCAdevint.dll
- ARCAdevlnk.dll
- ARCAliblog.dll

2.5.1 Example code

The parameters of all the functions are in JSON format, they are typically the parameters in input and in output of the functions.

If the file status.json is present in the same folder as where all the DLLs are located, then the output JSON structure will include the error code description as in the below Example

OutputData:

```
{
  "reply":
 {
    "replyCode": "0",
    "replyCodeDescription": "OK"
  }
}
```

If ARCAliblog.dll is present in the same folder as where ARCAdevint.dll is located, a tracefile for ARCAdevint and another one for ARCAdevlnk will be automatically produced.

To avoid the tracefiles to be created simply remove ARCAliblog.dll

ARCAdevint architecture supports multiple devices from the same instance returning a connection handle called hConn.

3 Functions

3.1 ARCAInitialize

Result API ARCAInitialize(**float** *versioning* , **char*** *inData*);
versioning: The value must be set to **LIBRARY_VERSION_1_0**. Less than 1.0 for alpha and beta versions.

inData:
{
“pathtrace” : “xxxx”
}
 	
Pathtrace = path where the trace file will be created

Description

First function that the application MUST call to start the communication with the device.

This function must be called only once

It accepts the trace pathname in the form of absolute path only. See example .

Strings not accepted: “./” ”.\” “..\” “../”

Return Value

AR_OKAY if successful.

AR_LIBRARY_INVALID_INITIALIZATION

Example

inData:

```
{
  "pathtrace": "C:\\ RECYCLERS \\RecyclerTool\\RecyclerTool\\bin\\Debug\\TRACE\\"
}
```

3.2 ARCAConnect

Result API ARCAConnect(**char** **inData*,
int *(*userFunc)(LSPSTR sEventInfo)*
int **hConnect,*
int *lloutData,*
char **outData)*);

inData:

```
{
        “cmdMode”: “xxx”,
        “connectionType”: “xxxx”,
        “encryption” : “xxx” ,
        “protocolSet”: “xxxx”,
        “TCPIPset”:
        {
              "address": “xxx.xxx.xxx.xxx”,
              "port": "xxxx"
        },
        “USBSerialNumber”: “xxxx”,
        “COMset”:
        {
              "COMname": “COMxxx”,
              "baudRate": "xxxx",
              "parity": "xxxx",
              "dataBits": "xxxx",
              "stopBits": "xxxx"
        },
}
```

Accepted values for:

cmdMode = mode execution, “SYNC” or “ASYNC”. (ASYNC commands available only with Novus). When in ASYNC mode the events listed hereafter are generated.
connectionType = “USB”, “LAN” or “RS232”
encryption = “SSL” if SSL is required “NO_ENC” if no encryption
protocolSet = “STANDARD” or “SIMPLIFIED”
TCPIPset = TCP/IP parameters.
address = device IP address. In this field It is also possible to pass a logical Name (hostname). In this case in Windows set the hostname in the file C:\Windows\System32\drivers\etc\hosts
In example:
172.19.12.42 CM18NS-N00001
port = connection port number.
USBSerialNumber = USB serial number.(put “0”)
COMname = COM port to connect.

COMset default values are:
baudRate = 9600
parity = None, Even or Odd
dataBits = 8
stopBits = 1

*int (*userFunc)(EVENT_INFO *sEventInfo :*

Call-back function called by ARCADevInt when an asynchronous event occurs.
If the application does not handle the asynchronous event, this parameter must be set to NULL.
The parameter in input to the userFunc is a JSON message like the following:

EVENTS:

```
{
  content : {"statusCode":xx,"statusDescription":"xxx"}
  event : {"description":"xx","type":"xx"}
}
```

The meaning of each field will follow this table:

Type	Area	statusCode
0	Error	Error code
1	Input bin	0 empty, 1 unexpected occupied, 2 occupied
2	Output bin	0 empty, 1 unexpected occupied, 2 occupied
3	Reject bin	0 empty, 1 unexpected occupied, 2 occupied
4	Upper transport	0 close, 1 open
5	Safe door	0 close, 1 open
6	Cassette Cage	0 close, 1 open
10	Recovery procedure	0 end, 1 start
11	Download procedure	0 end, 1 start
12	Warning	Warning code

lloutData (I)

Specifies *outData* size.
Advised 256

outData (O)

```
{
  “reply”:
  {
    "replyCode": "0",
    "replyCodeDescription": "OK"
  }
}
```

Description

This function must be invoked to establish the communication link with the specific device.

The function returns the connection handler to be used on all the subsequent functions.

If a UserFunction is passed to the function asynchronous events will be notified in the form of JSON messages. See Example section for details

Return Value

AR_OKAY if successful

AR_UNIT_NOT_FOUND

Refer to status.json for the full list and description of return codes.

Example

InData (USB)


```
{
 "connectionType": "USB",
 "protocolSet": "SIMPLIFIED",
 "TCPIPset": {
  "address": "172.19.12.199",
  "port": "8000"
 },
 "USBSerialNumber": "0",
 "COMSet": {
  "COMname": "COM6",
  "baudrate": "9600",
  "parity": "NONE",
  "dataBits": "8",
  "stopBits": "1"
 }
}
```

InData (LAN)


```
{
 "connectionType": "LAN",
 "protocolSet": "SIMPLIFIED",
 "TCPIPset": {
  "address": "172.19.12.197",
  "port": "8000"
 },
 "USBSerialNumber": "0",
 "COMSet": {
  "COMname": "COM6",
  "baudrate": "9600",
  "parity": "NONE",
  "dataBits": "8",
  "stopBits": "1"
 }
}
```


InData ("LAN with logical name)"


```
{
  "cmdMode":"SYNC",
  "connectionType":"LAN",
  "encryption":"NO_ENC",
  "protocolSet":"SIMPLIFIED",
  "TCPIPset":{
    "address":"CM18NS-N00001",
    "port":"8002"
  },
```

```
    "USBSerialNumber":"0",
    "COMSet":{
      "COMname":"COM2",
      "baudrate":"9600",
      "parity":"None",
      "dataBits":"8",
      "stopBits":1
    }
  }
```


InData (RS232)


```
  {
    "cmdMode":"SYNC",
    "connectionType":"RS232",
    "encryption":"NO_ENC",
    "protocolSet":"SIMPLIFIED",
    "TCPIPset":{
      "address":"172.19.12.196",
      "port":"8002"
    },
    "USBSerialNumber":"0",
    "COMSet":{
      "COMname":"COM2",
      "baudrate":"9600",
      "parity":"NONE",
      "dataBits":"8",
      "stopBits":1
    }
  }
```

EVENTS Examples:

{

content : {"statusCode":1,"statusDescription**":"unexpected input bin full"}**

event : {"description":"Input Bin","type":"1"}

}

{

content : {"statusCode":0,"statusDescription**":"input bin empty**"}

event : {"description":"Input Bin","type":"1"}

}

{

content : {"statusCode":1,"statusDescription":"**unexpected output bin full**"}

event : {"description":"Output Bin","type":"2"}

}

{

content : {"statusCode":0,"statusDescription":"**output bin empty**"}

event : {"description":"Output Bin","type":"2"}

}

{

content : {"statusCode":1,"statusDescription":"**unexpected reject bin full**"}

event : {"description":"Reject Bin","type":"3"}

}

{

content : {"statusCode":0,"statusDescription":"**reject bin empty**"}

event : {"description":"Reject Bin","type":"3"}

}

3.3 ARCADisconnect

Result API ARCADisconnect(int *hConnect*
char **inData*,
int *lloutData,*
char **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
    reply:
    {
        "replyCode": "xxxx" ,
        "replyCodeDescription": "xxxx"
     }
}.
```

Description

Close a connection between a client application and the device.

Return Value

AR_OKAY if successful.

AR_INVALID_HANDLE

3.4 ARCADeviceStatus

Result API ARCADeviceStatus(**int** *hConnect,*
char ***inData**,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"statusType": "xxxx"
}

Accepted values for "statusType":
"statusType" = "**extended**"
Using this command, it is possible to have a detailed information regarding the status of every module. This command can also be sent in an error condition since no previous 'Open' is required. No status-code is returned for missing modules.

"statusType" = "**pocketsAndCover**"
Check the status of all the unit covers/door/input/output slots, such as Safe door, top cover, input slot, Left extern button of CMCONS, Right extern button of CMCONS.

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)

in case of "extended"
{

```
"content":
    {
        "feeder":
        {
            "statusCode": "xxxxxx",
            "description": "xxx"
        },
```

```
“RTC”:
{
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“noteReader”:
 {
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“safeController”:
{
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteA”:
 {
      “cassetteCode": "x" ,
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteB”:
{
      “cassetteCode": "x" ,
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteC”:
 {
      “cassetteCode": "x" ,
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteD”:
 {
      “cassetteCode": "x" ,
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteE”:
{
      “cassetteCode": "x" ,
      “statusCode”: “xxxxxx”,
      “description”: “xxx”
},
“cassetteF”:
 {
```

```
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteG”:
                {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteH”:
                 {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteI”:
                 {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteJ”:
                {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteK”:
                {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
                “cassetteL”:
                {
                    “cassetteCode": "x" ,
                    “statusCode”: “xxxxxx”,
                    “description”: “xxx”
                },
        }
      reply:
       {
            "replyCode": "xxxx" ,
            "replyCodeDescription": "xxxx"
        }
}
```

In case of “pocketsAndCover”


```
{
      "reply":
      {
         "replyCode":"0",
         "replyCodeDescription":"not found"
      },
      "slotStatus":
      {
        "Cat2Boxpresent":"xx",
        "Cat2Boxstatus":"xx",
        "cover":"xx",
        "feeder":"xx",
        "inputSlot":"xx",
        "leftSlot":"xx",
        "leftexternbutton":"xx",
        "rejectSlot":"xx",
        "rightSlot":"xx",
        "rightexternalbutton":"xx",
        "safeDoor":"xx"
      }
}
```


Description

Check the status of all modules of the connected unit. For each module the internal low level status code will be returned

in Example:

40 : status OK

4: status full

5: status empty

The same status is described in "description" field.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData:


```
{
 "statusType": "pocketsAndCover"
}
```


outData:


```
{
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 },
 "slotStatus":
 {
  "Cat2Boxpresent": "present",
  "Cat2Boxstatus": "empty",
  "cover": "closed",
  "feeder": "closed",
  "inputSlot": "not empty",
  "leftSlot": "empty",
  "leftexternbutton": "ligth on",
  "rejectSlot": "empty",
  "rightSlot": "empty",
  "rightexternalbutton": "light off",
```

```
    "safeDoor": "closed"
   }
  }
inData:
  {
   "statusType": "extended"
  }

outData:
  {
   "content":
  {
    "RTC": {
     "description": "Status OK",
     "statusCode": "40"
    },
    "cassetteA": {
     "cassetteCode": "A",
     "description": "Drawer empty",
     "statusCode": "05"
    },
    "cassetteB": {
     "cassetteCode": "B",
     "description": "Drawer empty",
     "statusCode": "05"
    },
    "cassetteC": {
     "cassetteCode": "C",
```

"description": "Drawer empty",
"statusCode": "05"
},
"cassetteD": {
"cassetteCode": "D",
"description": "Status OK",
"statusCode": "40"
},
"cassetteE": {
"cassetteCode": "Z",
"description": "Drawer present but switched OFF",
"statusCode": "00"
},
"cassetteF": {
"cassetteCode": "F",
"description": "Status OK",
"statusCode": "40"
},
"cassetteG": {
"cassetteCode": "G",
"description": "Status OK",
"statusCode": "40"
},
"cassetteH": {
"cassetteCode": "H",
"description": "Drawer empty",
"statusCode": "05"
},

"cassetteI": {
"cassetteCode": "I",
"description": "Drawer empty",
"statusCode": "05"
},
"cassetteJ": {
"cassetteCode": "J",
"description": "Drawer empty",
"statusCode": "05"
},
"cassetteK": {
"cassetteCode": "K",
"description": "Status OK",
"statusCode": "40"
},
"cassetteL": {
"cassetteCode": "L",
"description": "Drawer empty",
"statusCode": "05"
},
"feeder": {
"description": "Status OK",
"statusCode": "40"
},
"noteReader": {
"description": "Status OK",
"statusCode": "40"
},

```
  "safeController": {
    "description": "Status OK",
    "statusCode": "40"
  }
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.5 ARCAOpen

Result API ARCAOpen(**int** *hConnect,*
char ***inData*,
int *lloutData,*
char **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"opSide": "xxxx",
"password": "xxxx",
}

Accepted values for:
opSide = (A-T)
For the moment only **L** and **R** are allowed in NOvus
password = default value "123456"

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

Reserve exclusive control of the device to the calling process.
An application must call this function to reserve the device before issuing the operation of Deposit Withdrawal or Counting.
The specified OpSide (either L(eft) or R(ight)) light on the TCR cabinet will be switched on with a green light (novus). The other side lamp will be switched on with a red light (novus)

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  "opSide": "L",
  "password": "123456"
}
```

outData:

```
{
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.6 ARCAClose

Result API ARCAClose(int *hConnect,* **char** ***inData**, **int** *lloutData,* **char** **outData)*;
hConnect (I) Handle returned by ARCAConnect.
inData (I) Must be a no NULL pointer, no input json data
lloutData (I) Specifies *outData* size. Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}

Description

The calling application release the exclusive control of the device when the current operation is finished.

Return Value

AR_OKAY

AR_OPEN_NOT_DONE

3.7 ARCAgetCashData

Result API ARCAgetCashData(**int** *hConnect,*
char ***inData*,
int *lloutData,*
char **outData);*

hConnect (I)

Handle returned by ARCAConnect.

inData (I)

Must be a no NULL pointer, no input json data to retrieve the physical cashdata of the machine, or:

{
"noteType" : " xxxx"
}

In case you specifiy "noteType" = "**unfitSafeLastDeposit**" it is possible to know detail of UNFIT notes deposited in safe during last deposit transaction.

In case you specify "noteType" = "**unfitLastDeposit**" it is possible to know detail of unfit notes of last deposit transaction

In case you specify "noteType" = "**suspectNotAuthenticated**" is possible to know the detail of suspect (Cat 2 ECB) and not authenticated (Cat 3 ECB) notes of the last deposit transaction sorted by denomination of reader

In case you specify "noteType" = "**suspect"** is possible to know the detail of suspect (Cat 2) notes of the last deposit transaction, sorted by denomination of reader

In case you specifit "noteType" = "**unfitClassificationDetails**", is possible to obtain the causal of unfit banknotes found in last deposit operation; in this case you can pass the number of banknotes to know the detail of the banknote noteNum=" xxxx"

In case you specify "noteType" = "**notAuthenticated**" is possible to know the detail of not authenticated (Cat 3 ECB) notes of the last deposit transaction sorted by denomination of reader

In case you specify “notetype” = “**unfitOutputLastDeposit**” it is possible to know detail of UNFIT notes present on the output slot during last deposit transaction.

In case you specify “noteType” = “**advanced**” **(Only for this parameter no previous ARCAOpen necessary**)
The operator will be able to know: which cassettes are enabled to the specified side and the specified password, which cassette has been switched-off, the cassette status, the note denomination for each cassette, the number of notes contained in every cassette, the average number of notes that can still be loaded

In this case you must specify parameters:

- “opSide” : “xxx” es : “L”
- “password” : “xxxx” es : “12345”

lloutData (I)

Specifies *outData* size.
Advised 4096

outData (O)

In case of no input inData

```
{
  reply: {
            "replyCode": "xxxx",
            "replyCodeDescription": "xxxx"
            }

 "content": {
   “totalNotes”: “nnn”,
   “noteReportElements”: “nnn”,
   “noteReport”: [
     { “cassetteID”: “x”, “noteID”: “xxxx”, “numberNote”: “nnn”, “capacity”:
            “nnn”, “switch”: “xxx”, “detailsCassette”: {“CPxx”: “nnn”, “CPxx”:
            “nnn” } },
     { “cassetteID”: “x”, “noteID”: “xxxx”, “numberNote”: “nnn”, “capacity”:
            “nnn”, “switch”: “xxx”, “detailsCassette”: {“CPxx”: “nnn”, “CPxx”:
            “nnn” } },
     { “cassetteID”: “x”, “noteID”: “xxxx”, “numberNote”: “nnn”, “capacity”:
            “nnn”, “switch”: “xxx”, “detailsCassette”: {“CPxx”: “nnn”, “CPxx”:
            “nnn” } },
     …
   ]
   }
}
```

In case of . “noteType” advanced


```
{
  "content":{
    "noteReport":[
      {
        "capacity":"xx",
        "cassetteID":"xx",
"cassetteIDstatus":"xx",
        "noteID":"xxx",
        "numberNote":"xx",
        "side_enabled":"xx",
        "switch":"xx"
                }
 ….
 ],
 "noteReportElements":"12",
    "totalNotes":"17"
  }"reply":{
    "replyCode":"0",
    "replyCodeDescription":"OK"
  }
```


In case of “noteType” = “unfitSafeLastDeposit” ,”unfitLastDeposit”, “suspectNotAuthenticated” , “suspect” , “unfitClassificationDetails” , “notAuthenticated”


```
{
        “replyCode”: “xxxx” ,
“replyCodeDescription”: "xxxx”

"content": {
        "NoteReportElements": "xxxx”
        {
                "NoteReport": [{
                        "noteType": "xxxx",
                        "numberNote": "xxxx"
                        }{
                "noteType": "xxxx",
                "numberNote": "xxxx"
                }],
        }
}
```

Description

This function should be used to retrieve the current count of notes stored in each recycle cassette. The data returned is the count of notes physically stored in each cassette.

This command needs ARCAOpen to be issued before excepting “advanced”

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData = **NULL**

outData


```
{
  "content": {
    "noteReport": [
      {
        "capacity": "395",
        "cassetteID": "A",
        "noteID": "AKC*",
        "numberNote": "0",
        "switch": "enabled"
      },
      {
        "capacity": "395",
        "cassetteID": "B",
        "noteID": "AKBB",
        "numberNote": "0",
        "switch": "enabled"
      },
      {
        "capacity": "395",
        "cassetteID": "C",
        "noteID": "AKD*",
        "numberNote": "0",
        "switch": "enabled"
```

},
{
"capacity": "395",
"cassetteID": "D",
"noteID": "AKE*",
"numberNote": "0",
"switch": "enabled"
},
{
"capacity": "293",
"cassetteID": "E",
"noteID": "CPE*",
"numberNote": "76",
"switch": "enabled"
},
{
"capacity": "330",
"cassetteID": "F",
"noteID": "CPD*",
"numberNote": "60",
"switch": "enabled"
},
{
"capacity": "279",
"cassetteID": "G",
"noteID": "CPG*",
"numberNote": "71",
"switch": "enabled"

```
},
{
  "capacity": "330",
  "cassetteID": "H",
  "detailsCassette": {
    "CPCA": "1",
    "CPDA": "1",
    "CPEA": "0",
    "CPGA": "0",
    "CPHA": "0",
    "CPIA": "0",
    "CPJA": "1"
  },
  "noteID": "CP--",
  "numberNote": "3",
  "switch": "enabled"
},
{
  "capacity": "0",
  "cassetteID": "I",
  "noteID": "0000",
  "numberNote": "0",
  "switch": "disabled"
},
{
  "capacity": "0",
  "cassetteID": "J",
  "noteID": "0000",
```

"numberNote": "0",
"switch": "disabled"
},
{
"capacity": "0",
"cassetteID": "K",
"noteID": "0000",
"numberNote": "0",
"switch": "disabled"
},
{
"capacity": "0",
"cassetteID": "L",
"noteID": "0000",
"numberNote": "0",
"switch": "disabled"
},
{
"capacity": "0",
"cassetteID": "M",
"noteID": "0000",
"numberNote": "0",
"switch": "disabled"
},
{
"capacity": "0",
"cassetteID": "N",
"noteID": "0000",

```
      "numberNote": "0",
      "switch": "disabled"
     },
     {
      "capacity": "0",
      "cassetteID": "O",
      "noteID": "0000",
      "numberNote": "0",
      "switch": "disabled"
     },
     {
      "capacity": "0",
      "cassetteID": "P",
      "noteID": "0000",
      "numberNote": "0",
      "switch": "disabled"
     }
    ],
    "noteReportElements": "16",
    "totalNotes": "210"
   },
   "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
   }
  }
```

inData


```
{
  "noteType": "advanced",
  "opSide": "L",
  "password": "123456"
}
```


outData


```
{
  "content": {
    "noteReport": [
      {
        "capacity": "395",
        "cassetteID": "A",
        "cassetteIDstatus": "05",
        "noteID": "AKC*",
        "numberNote": "0",
        "side_enabled": "Y",
        "switch": "1"
      },
      {
        "capacity": "395",
        "cassetteID": "B",
        "cassetteIDstatus": "05",
        "noteID": "AKBB",
        "numberNote": "0",
        "side_enabled": "Y",
        "switch": "1"
      },
      {
        "capacity": "395",
```

"cassetteID": "C",
"cassetteIDstatus": "05",
"noteID": "AKD*",
"numberNote": "0",
"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "395",
"cassetteID": "D",
"cassetteIDstatus": "05",
"noteID": "AKE*",
"numberNote": "0",
"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "293",
"cassetteID": "E",
"cassetteIDstatus": "40",
"noteID": "CPE*",
"numberNote": "76",
"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "330",
"cassetteID": "F",
"cassetteIDstatus": "40",
"noteID": "CPD*",
"numberNote": "60",

"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "279",
"cassetteID": "G",
"cassetteIDstatus": "40",
"noteID": "CPG*",
"numberNote": "71",
"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "330",
"cassetteID": "H",
"cassetteIDstatus": "40",
"noteID": "CP--",
"numberNote": "3",
"side_enabled": "Y",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "I",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{

"capacity": "0",
"cassetteID": "J",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "K",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "L",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "M",
"cassetteIDstatus": "00",
"noteID": "0000",

"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "N",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "O",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
},
{
"capacity": "0",
"cassetteID": "P",
"cassetteIDstatus": "00",
"noteID": "0000",
"numberNote": "0",
"side_enabled": "N",
"switch": "1"
}

```
      ],
      "noteReportElements": "16",
      "totalNotes": "210"
    },
    "reply": {
      "replyCode": "0",
      "replyCodeDescription": "OK"
    }
  }
```


inData:


```
  {
    "noteType": "unfitLastDeposit"
  }
```


outData:


```
  {
    "content": {
      "NoteReport": [
        {
          "noteType": "CPGA",
          "numberNote": "1"
        }
      ],
      "NoteReportElements": "1"
    },
    "reply": {
      "replyCode": "0",
      "replyCodeDescription": "OK"
    }
  }
```

inData:


```
{
  "noteType": "unfitSafeLastDeposit"
}
```


outData:


```
{
  "content": {
    "NoteReport": [
      {
        "noteType": "CPGA",
        "numberNote": "1"
      }
    ],
    "NoteReportElements": "1"
  },
  "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
  }
```

3.8 ARCADepositStart

Result API ARCADepositStart (**int** *hConnect,*
char ***inData**,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data
or
“manageLights” : "true"

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
    reply:
    {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
     }
}.
```

Description

In case of no InputData

The purpose of this functions is to start a multiple deposit operation session. While the session is in progress the application calls the **ARCADeposit()** causes to stores the progressive banknote count of the single deposit operation.

The application needs to confirm (complete) a deposit session by using the **ARCADepositEnd()** function with the parameter *Accept*.

If application needs to cancel the session it must call the **ARCADepositEnd ()** function with the parameter *Reject* and the unit will automatically dispense all notes deposited after **ARCADepositStart ()** call.

In case of “manageLights” : "true"

This command is used to switch-on input slot’s led.

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.9 ARCADeposit

Result API ARCADeposit(int *hConnect,*
char ***inData**,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"cmdMode": "xxx",
"casseteDetails" : "xxxx" ,
"mode" : "xxxx"
}

Accepted values for:
cmdMode = deposit mode execution, "SYNC" or "ASYNC".
ASYNC for future enhancement
casseteDetails = if true shows the complete detail of cassette in case of multi-denomination cassettes(detailsBefore, detailsAfter)
mode = if not present the command execute the standard deposit

if **mode** parameter is present, it indicates the type of deposit :

- "**nation**" -> This mode is used when on the unit are present cassettes with different nation. For example: EURO for first 4 cassette and CANADA for the other 4 cassette so on the Reader the reference denomination are EURO in bank 1 and CANADA in bank 2. With this mode the CM18/CM18T accept only the denomination specified in n.id(denomination for example "EU00" is valid only the first two characters) and select automatically the bank on the reader. In this case it is necessary to specify **denomID** : "xxxx"
- **"continuousFeeding"** -> To Start deposit in Continuous feeding, it is necessary to specify also:
  "**continuousFeedingAction**" : "start"
  "**continuousFeedingTime**" : "5000" -> 5000 (5 *sec*) and above = send unsolicited event each xxxx msecs

  The application must send a "st*op"* deposit in Continuous feeding in order to retrieve the operation result.
  To stop deposit in Continuous feeding, is necessary to specify:
  continuousFeedingAction : "stop"

lloutData (I)

Specifies *outData* size.
Advised 2048

outData (O)

If the parameter *mode* set to “SYNC” the following JSON structure will be returned:

```
{
        “reply”:
        {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
        }

        "content":
        {
          “depositReport”:
          {
              “totalNoteDep”: “xxx”,
              “refused”: “xxx”,
              “unrecognized”: “xxx”,
          },
          “noteReportElements”: “xxx”,
          “noteReport”: [
            { “noteType”: “xxxx”, “numberNote”: “nnn” },
            { “noteType”: “xxxx”, “numberNote”: “nnn” },
            { “noteType”: “xxxx”, “numberNote”: “nnn” },
          ]
          “safeNumberOfCassettes”: “xxx”,
          “safeReport”: {
            "0": {
              “storageID”: “x”,
              "storageConf": "xxxx",
              "noteQty": "nnn"
              “detailsBefore”: {
                “xxxx”: “xxx”,
                “xxxx”: “xxx”,
                …
              }
              “detailsAfter”: {
                “xxxx”: “xxx”,
                “xxxx”: “xxx”,
                …
              }
            },
```

```
                    "xx": {
                      “storageID”: “x”,
                      "storageConf": "xxxx",
                      "noteQty": "nnn"
                      “detailsBefore”: {
                        “xxxx”: “xxx”,
                        “xxxx”: “xxx”,
                        …
                      }
                      “detailsAfter”: {
                        “xxxx”: “xxx”,
                        “xxxx”: “xxx”,
                        …
                      }
                    },
                    ….
                  }
              }
}
```

Returned value:
depositReport = Deposit report structure field description.
totalBankDep = Number of banknotes sent to the cassettes (or counted in the output bay).
refused = Number of notes rejected (for doublefeeding, or notes that do not have an assigned cassette or the cassette is full or not operational)
unrecognized = Number of rejected notes, suspect/counterfeit notes, unrecognized notes or notes that do not have an assigned cassette or the cassette is full or not operational.
noteReportElements = number of noteReport structures.
noteReport = Note report structure field description.
noteType = specifies the banknote denomination.
numberNote = specifies the number of banknotes.
cassettesReport = details the banknotes present in the cassettes.
noteType = specifies the banknote denomination.
storageID = cassette ID
detailsBefore = number of banknotes present in the cassette before the deposit .
detailsAfter = number of banknotes present in the cassette after the deposit.

Description

Start banknotes deposit.

If the parameter cassetteDetail in inData is set to true, the function returns the content of each single cassette (notes details) before and after the deposit operation.

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData:


```
{
 "cmdMode": "SYNC",
 "cassetteDetails": "false"
}
```


outData:


```
{
 "content": {
  "depositReport": {
   "refused": "0",
   "totalNoteDep": "1",
   "unrecognized": "0"
  },
```

"noteReport": [
{
"noteType": "CPDA",
"numberNote": "1"
}
],
"noteReportElements": "1",
"safeNumberOfCassettes": "12",
"safeReport": {
"0": {
"noteQty": "0",
"storageConf": "BOXA",
"storageID": "A"
},
"1": {
"noteQty": "0",
"storageConf": "BOXB",
"storageID": "B"
},
"10": {
"noteQty": "0",
"storageConf": "CPIA",
"storageID": "K"
},
"11": {
"noteQty": "0",
"storageConf": "CPJA",
"storageID": "L"

},
"2": {
"deposited": {
"CPDA": "1"
},
"noteQty": "1",
"storageConf": "CP--",
"storageID": "C"
},
"3": {
"noteQty": "0",
"storageConf": "CP++",
"storageID": "D"
},
"4": {
"noteQty": "0",
"storageConf": "CP??",
"storageID": "E"
},
"5": {
"noteQty": "0",
"storageConf": "CPCA",
"storageID": "F"
},
"6": {
"noteQty": "0",
"storageConf": "CPDA",
"storageID": "G"

```
    },
    "7": {
      "noteQty": "0",
      "storageConf": "CPEA",
      "storageID": "H"
    },
    "8": {
      "noteQty": "0",
      "storageConf": "CPGA",
      "storageID": "I"
    },
    "9": {
      "noteQty": "0",
      "storageConf": "CPHA",
      "storageID": "J"
    }
   }
  },
  "reply": {
   "replyCode": "0",
   "replyCodeDescription": "OK"
  }
}
```

continuousFeeding example:

inData

{

cassetteDetails : "false"

cmdMode : "SYNC"

continuousFeedingAction : "start"

continuousFeedingTime : "5000"

mode : "continuousFeeding"

}

outData

{

Content: {

"depositReport":{

"refused":"",

"totalNoteDep":"",

"unrecognized":""

},

"noteReportElements":"0",

"safeNumberOfCassettes":"0"

}

reply : {"replyCode":"0","replyCodeDescription":"OK"}

}

inData

{

cassetteDetails : "false"

cmdMode : "SYNC"

continuousFeedingAction : "stop"

mode : "continuousFeeding"

}

outData


```
      "2":{
        "noteQty":"0",
        "storageConf":"CPE*",
        "storageID":"C"
      },
      "3":{
        "noteQty":"0",
        "storageConf":"CPH*",
        "storageID":"D"
      },
      "4":{
        "noteQty":"0",
        "storageConf":"CPG*",
        "storageID":"E"
      },
      "5":{
        "noteQty":"0",
        "storageConf":"CPI*",
        "storageID":"F"
      },
      "6":{
        "noteQty":"0",
        "storageConf":"CPJ*",
        "storageID":"G"
      },
      "7":{
        "noteQty":"0",
        "storageConf":"CP--",
        "storageID":"H"
      }
    }
  }"reply":{
    "replyCode":"0",
    "replyCodeDescription":"OK"
  }
}
```

3.10 ARCADepositEnd

Result API ARCADepositEnd(**int** *hConnect,*
char ***inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
“reviewType”: “xxx”,
or
“manageLights” : "true"

}
Accepted values:
reviewType = **accept/list/undo**

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
{
“reply”: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content": {
“totalNotesProcessed”: “xxx”,
“noteReport”: [
{ “noteType”: “xxxxxx”, “numberNote”: “xxx” },
{ “noteType”: “xxxxxx”, “numberNote”: “xxx” },
{ “noteType”: “xxxxxx”, “numberNote”: “xxx” },
]
}
}

Returned values:

totalNotesProcessed = total number of banknotes processed.
noteReport = Note report structure field description.
cassetteID = cassette identifier
noteType = specifies the banknote denomination
numberNote = specifies the number of banknotes

Description

In case of "manageLights" : "true"

This command is used to switch-off input slot's led.

In case of reviewType = accept /undo

This command is used to confirm deposit or rollback

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata


```
{
 "cmdMode": "SYNC",
 "reviewType": "undo"
}
```


Outdata


```
{
 "noteReport": [
  {
   "noteType": "CPCA",
   "numberNote": "1"
  }
 ],
 "noteReportElements": "1",
 "reply": {
  "replyCode": "210",
  "replyCodeDescription": "Denomination empty"
 }
}
```


Indata:


```
{
 "cmdMode": "SYNC",
 "reviewType": "accept"
```

```
}
```

Outdata:

```
{
 "noteReport": [
  {
   "noteType": "CPCA",
   "numberNote": "1"
  },
  {
   "noteType": "CPDA",
   "numberNote": "1"
  },
  {
   "noteType": "CPEA",
   "numberNote": "1"
  }
 ],
 "noteReportElements": "3",
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

Indata:

```
{
 "cmdMode": "SYNC",
 "reviewType": "list"
}
```

Outdata:


```
{
 "noteReport": [
  {
   "noteType": "CPDA",
   "numberNote": "1"
  }
 ],
 "noteReportElements": "1",
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.11 ARCACount

Result API ARCACount(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"cmdMode": "xxx",
"mode" : "xxx"

Optional
"boundleSize": "xxx",
"boundleCountingMode": "xxx",
"denomID" : "xxx" **,**

~~"class" : "xxx"~~ **for future enhancement**
~~"way" : "xxx"~~ **for future enhancement**
~~"session" : "xxx"~~ **for future enhancement**
~~"cat" : "xxx"~~ **for future enhancement**
~~"fit" : "xxx"~~ **for future enhancement**
~~"note" : "xxx"~~ **for future enhancement**
}

cmdMode = count mode execution, "SYNC" or "ASYNC".
ASYNC for future enhancement

In case you specifiy **mode** = "**standardCount**" the standard counting is executed
No parameter needed

In case you specifiy **mode** = "**modeBlockCount**" The notes are all directed to exit compartment. The reply sent to the host will contain the number of notes for each denomination. In case of an unknown bank note or in case of JAM the procedure will stop and the reply will tell how many notes at the end of the bundle have not been recognized.

Parameter needed for:
boundleSize = Number of banknotes per boundle, Range accepted : 0 to 100.
0: max number of notes (100)
boundleCountingMode = value accepted:
StartCounting - Start a new counting transaction.

ContinueCounting - Continue the previous counting transaction, valid after a StartCounting.
FullCounting – Start a new counting transaction, feed all the banknotes present in the input slot and split the banknotes in boundles according to the **boundleSize** parameter.

In case you specifiy mode = **“singleEmis”** or “**multiEmis**”

This command is used for deposit counting fit/unfit batch. During the deposit, the notes fit equal to the specified reference note are put in output slot and all others notes are put in the reject slot. It stop when bundle size is reached in output slot or when there are no more input note or the reject slot is full (100 notes).

Parameter needed for:
boundleSize = Number of banknotes per boundle, Range accepted : 1 to 200.
boundleCountingMode = value accepted:
StartCounting - Start a new counting transaction.
ContinueCounting - Continue the previous counting transaction, valid after a StartCounting.

In case of ContinueCounting you must specify the parameter “denomID” to continue validate the specific denomination

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)

```
{
        “reply”:
        {
          "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
              }
        "content":
        {
          “countReport”:
          {
            “totalNoteCounted”: “xxx”,
            “refused”: “xxx”,
            “unrecognized”: “xxx”,
          },
          “noteReportElements”: “xxx”,
          “noteReport”: [
            { “noteType”: “xxxxxx”, “numberNote”: “xxx” },
            { “noteType”: “xxxxxx”, “numberNote”: “xxx” },
            { “noteType”: “xxxxxx”, “numberNote”: “xxx” },
```

```
                    ]
                }
}
```

Returned values:

depositReport = Deposit report structure field description.
totalBankDep = Number of the banknotes sent to the cassettes (or counted in the output bay).
refused = Number of notes rejected for doublefeeding, or notes that do not have an assigned cassette or the cassette is full or not operational.
unrecognized = Number of rejected notes, suspect/counterfeit notes, unrecognized notes or notes do not have an assigned cassette or the cassette is full or not operational.
noteReportElements = number of noteReport structures.
noteReport = Note report structure field description.
noteType = specifies the banknote denomination.
numberNote = specifies the number of banknotes.

Description

Perform the counting operation. The notes are all directed to the Output (FIT and UNFIT) and Reject (not Recognized, Suspect, Not Authenticated) compartment. The reply sent to the host will contain the number of notes for each denomination.

If **boundleSize** and **boundleCountingMode** are not set the unit count all the notes present in the input slot.

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata


```
{
 "cmdMode": "SYNC",
 "boundleSize": "0",
 "boundleCountingMode": "StartCounting"
}
```


outdata


```
{
 "content": {
  "countReport": {
   "refused": "1",
   "totalNoteCounted": "2",
   "unrecognized": "0"
  },
  "noteReport": [
   {
    "noteType": "CPCA",
    "numberNote": "1"
   },
   {
    "noteType": "CPDA",
    "numberNote": "1"
   }
  ],
  "noteReportElements": "2"
```

```
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```


inData


```
{
  boundleCountingMode : "StartCounting"
  boundleSize : "100"
  cmdMode : "SYNC"
  mode : "singleEmis"
}
```


inData


```
{
  boundleCountingMode : "ContinueCounting"
  boundleSize : "100"
  cmdMode : "SYNC"
  denomID : "CPGA"
  mode : "singleEmis"
}
```

3.12 ARCADispense

Result API **ARCADispense**(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"cmdMode": "xxx",
"noteDispenseElements": "xxx",
"noteDispense": [
{ "denominationID": "xxxx", "numberNote": "nnn" },
{ "denominationID": "xxxx", "numberNote": "nnn" },
{ "denominationID": "xxxx", "numberNote": "nnn" },
….
]
or
"noteDispense": [
{ "cassetteID": "x", "numberNote": "nnn" },
{ "cassetteID": "x", "numberNote": "nnn" },
{ "cassetteID": "x", "numberNote": "nnn" },
….
]
alternateDispense : " tt **"**
}

Accepted values for:
cmdMode = dispense mode execution, "SYNC" or "ASYNC".
ASYNC for future enhancement

noteDispenseElements = number of noteDispense structures.
noteDispense = Note report structure field description.
denominationID = specifies the banknote denomination.
cassetteID = specifies the ID of the cassette from where the banknote will be taken.
numberNote = specifies the number of banknotes dispensed.
alternateDispense (only for Novus) = This command is also called fast empty and it is used to deliver the amount requested by the operator to both output slot (output and reject). If the cassette does not have enough notes the delivery will stop with

the 'Denomination Empty' warning. If you want to receive event you must connect to the device in **asynchronous mode**.

"tt" **:** Interval between unsolicited Get Cash Count events updates in **milliseconds**. The objective of this parameter is to allow the device to send notifications about the progress of the dispense operation.

lloutData (I)

Specifies *outData* size.
Advised 4096

outData (O)

If the parameter *mode* given in input is set to "ASYNC" this field must be set to NULL.

```
{
        "reply":
        {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
        }
        "content":
        {
          "totalNotesDispensed": "xxx",
          "noteReportElements": "xxx",
          "noteReport": [
            { "denominationID": "xxxx", "numberNote": "nnn" },
            { "denominationID": "xxxx", "numberNote": "nnn" },
            { "denominationID": "xxxx", "numberNote": "nnn" },
          ]

          or
          "noteReport": [
            { "cassetteID": "x", "numberNote": "nnn" },
            { "cassetteID": "x", "numberNote": "nnn" },
            { "cassetteID": "x", "numberNote": "nnn" },
            ….
          ]
                }
}
```

Returned value:
totalNotesDispensed = Total number of banconotes dispensed.
noteReportElements = number of noteReport structures.
noteReport = Note report structure field description.
denominationID = specifies the banknote denomination.
cassetteID = specifies the ID of the cassette from which the banknotes are taken.
numberNote = specifies the number of banknotes.

Description

Dispenses the requested number of banknotes.

The application can choose whether to specify the banknotes denomination to be dispensed or specify the drawer from which it will be extracted. Only one choice can be done, no mix.

If the dispense is per denomination it is mandatory to specify exactly the denomination assigned to the cassette.

In example you cannot request EUC* in the dispense command if the cassette is assigned to EUCB.

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata


```
{
 "cmdMode": "SYNC",
 "noteDispenseElements": "2",
 "noteDispense": [
  {
   "cassetteID": "H",
   "numberNote": "1"
  },
  {
   "cassetteID": "K",
   "numberNote": "1"
  }
 ]
}
```

Outdata


```
{
 "content": {
  "noteReport": [
   {
    "cassetteID": "H",
    "numberNote": "1"
   },
   {
    "cassetteID": "K",
    "numberNote": "1"
   }
  ],
  "noteReportElements": "2",
  "totalNotesDispensed": "2"
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.13 ARCAgetDeviceDateTime

Result API ARCAgetDeviceDateTime(**int** *hConnect*
char **inData*,
int *lloutData,*
char **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
        reply: {
                 "replyCode": "xxxx" ,
                 "replyCodeDescription": "xxxx"
                 }
        "content":
                {
                “seconds”: “nn”,
                “minutes”: “nn”,
                “hours”: “nn”,
                “weekDay”: “nn”,
                “day”: “nn”,
                “month”: “nn”,
                “year”: “nnnn”,
                }
}
```

Description

Get the device date and time.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.14 ARCAsetDeviceDateTime

Result API ARCAsetDeviceDateTime(**int** *hConnect*
char **inData*,
int *lloutData,*
char **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
“seconds”: “nn”,
“minutes”: “nn”,
“hours”: “nn”,
“weekDay”: “nn”,
“day”: “nn”,
“month”: “nn”,
“year”: “nnnn”,
}

Accepted values for:
seconds = Number of seconds to set, (00 -59).
minutes = Number of minutes to set, (00 - 59).
hours = Number of hours to set, (00 - 23).
weekDay = Week day to be set, (Monday, Tuesday, …).
day = Day number to set, (01 - 31).
month = Month number to set, (01 - 12).
year = Year number to set, 4 digits.

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
“reply”:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

Set the date and time of the device.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData:


```
{
 "seconds": "22",
 "minutes": "13",
 "hours": "13",
 "weekDay": "Tuesday",
 "day": "08",
 "month": "03",
 "year": "2022"
}
```


outData:


```
{
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.15 ARCAexeCassetteEmpty

Result API ARCAexeCassetteEmpty(**int***hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"cmdMode": "xxx",
"cassetteID": "xxx",
"password": "xxxx",
"outBoundleNotes": "xxx",
}

Accepted values for:
cmdMode = empty mode execution, "SYNC" or "ASYNC".
ASYNC for future enhancement

CassetteID = Physical cassette identifier (A…L).
password = default value "123456", same password given to the Open command.
outBoundleNotes = Specifies the size of the boundle of banknotes in output, Range - 50 to 200, default 100.

lloutData (I)
Specifies *outData* size.
Advised 512

outData (O)

{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}

```
                "content":
                {

                   “operationCode”: “xxx”,
                   “startingNotes”: “xxx”,
                   “outputNotes”: “xxx”,
                }.
}.
```

Returned value:
operationCode = Code returned in the response:
0 = ok
2 = cassette not present
4 = wrong password.
startingNotes = number of the logical banknotes counter in the cassette.
outputNotes = number of banknotes physically dispensed.

Description

Dispenses all banknotes stored inside the target cassette

The command terminates when the notes are removed from the output pocket.

This command needs ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata:


```
{
 "cmdMode": "SYNC",
 "cassetteID": "L",
 "password": "123456",
 "outBoundleNotes": "100"
}
```


Outdata:


```
{
 "content": {
  "operationCode": "0",
  "outputNotes": "1",
  "startingNotes": "1"
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.16 ARCAgetDeviceVersion

Result API ARCAgetDeviceVersion(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

{
"versionType": "xxx"
}

Accepted values for versionType :

- *UnitIdentification*
- *Options*
- digitalDistribution

- *rtc*
- *reader*
- *fpga*
- *safe*
- *fpga*
- *osc (only for novus)*
- *cassetteA-L*
- controller_name
- suite_name
- alarm
- ph01*(only for novus)*
- ph02*(only for novus)*
- usbModuleA
- usbModuleB
- *cassetteSensorBoardA-L(only for novus)*
- osVersion*(only for novus)*
- assetVersion*(only for novus)*
- cdfName

UnitIdentification : it is possible to get the identification code of the device

serial protocol.

Options : with this command it is possible to know the machine name(CM name (4 ASCII char) , Module compatibility software , Module compatibility hardware , Module Serial Number

otherParameters : it is possible to know the module name, firmware release, firmware version and the serial number of each single module present in the machine

lloutData (I)

Specifies *outData* size.
Advised 1024

outData (O)

```
{
    "reply":
    {
        "replyCode": "xxxx" ,
        "replyCodeDescription": "xxxx"
    }
    "content":{

    In case of UnitIdentification
    {
        "protocolInterface" : " xxxx"
    }

    In case of Options
    {
            "name": "xxxx",
            "ver_hw": "xxxx",
            "ver_sw": "xxxx",
            "serial_num": "xxxxxx"
    }
  In case of  digitalDistribution
 {
  digitalDistribution":"XXXXXXXXXXXX"
 }

 In case of  cdfName
 {
  cdfName":"XXXXXXXXXXXX"
 }


 In other cases
```

```
                {
                                “name”  : “xxxx” ,
                                “ver” : “xxxx” ,
                                “rel” , “xxxx”
                                “serial_num” , “xxxx”
                }
}
```

Description

With this command it is possible to get the identification of the unit and the version of the single module

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData:

```
{
 "versionType": "rtc"
}
```

outData:

```
{
 "content": {
  "rtc": {
   "name": "MC19",
   "rel": "0022",
   "serial_num": "555555555888",
   "ver": "5_04"
  }
 },
 "reply": {
```

```
    "replyCode": "0",
    "replyCodeDescription": "OK"
   }
  }
```


inData:


```
  {
   "versionType": "cassetteA"
  }
```


outData:


```
  {
   "content": {
    "cassetteA": {
     "name": "CR37",
     "rel": "B051",
     "serial_num": "207050096942",
     "ver": "1_00"
    }
   },
   "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
   }
  }
```


inData:

{
"versionType": "**UnitIdentification**"
}

outData:

{
"content": {
"protocolInterface": "CM28"
},
"reply": {
"replyCode": "0",
"replyCodeDescription": "OK"
}
}

inData:

{
"versionType": "**Options**"
}

outData:

{
"content": {
"name": "USF0",
"serial_num": "555555555888",
"ver_hw": "0H0B",
"ver_sw": "F000"
},
"reply": {

```
            "replyCode": "0",
            "replyCodeDescription": "OK"
          }
        }
```


inData


```
{
  versionType : "digitalDistribution"
}
```


outData


```
{
  content : {
        "digitalDistribution":"FF_3456"
  }
  reply :
  {
        "replyCode":"0",
        "replyCodeDescription":"OK"
  }
}
```

3.17 ARCAgetDeviceType

Result API ARCAgetDeviceType(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"type": "xxx"
}

Accepted values for type: **machine , machineExtended**

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)

{
"reply":
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content": in case of **machine**
{
"machine" : "xxxx" ,
"version": "xxx"
}
"content" in case of **machineExtended**
{
"led":"xxx",
"mode":"xxx",
"name":"xxx",

"numBag":"xxx",
"numCD80":"xxx",
"numCassette":"xxx",
"numProtoCassette":"xxx",
"numProtoBag":"xxx",
"protocolInterface":"xxx",
"type":"xxxx"
}
}

Returned value:
protocolInterface = possobile value CM25 CM27 CM28.
name = name (4 ASCII char) where: Ascii 1 CM type: T : Tower model U: Under table model Ascii 2 Banknote identifier type S : short edge recognition Ascii 3 Number of recycling cassette Ascii 4 Number of non-recycling cassette
type= Possible values are: CM20 CM18 CM18T CM18E CM20E CM20S CM20T CM18Solo
numProtoCassette= Number of configured cassettes used by protocol
numCassette= Number of configured cassettes
numCD80= Number of CD80 cassettes
mode= Cat 2 box presence. Possible values are: TARM CRM
led= Led type Possible values are: STD EVO
numBag= Number of of configured bag
numProtoBag= Number of configured bag used by protocol

Description

With this command it is possible to get the identification code of the device.

With “machineExtended” it is possible to know detailed setup of the unit: Unit identification, Protocol cassette number

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

In case of “machine”

inData

```
{
 "type": "machine"
}
```

outData

```
{
 "content": {
  "machine": "CM18T",
  "version": "STD"
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

For Novus:

outData

```
{
 "content": {
  "machine": "CM18TNS",
  "version": "STD"
 },
 "reply": {
  "replyCode": "0",
```

```
        "replyCodeDescription": "OK"
      }
    }
```

In case of “machineExtended”

inData

```
    {
      type : "machineExtended"
    }
```

outData

```
{
  "content":{
    "led":"STD",
    "mode":"CRM",
    "name":"US90",
    "numBag":"0",
    "numCD80":"0",
    "numCassette":"12",
    "numProtoCassette":"12",
    "numpProtoBag":"0",
    "protocolInterface":"CM28",
    "type":"CM18TNS"
  },
  "reply":{
    "replyCode":"0",
    "replyCodeDescription":"OK"
  }
}
```

3.18 ARCAexeDeviceReset

Result API ARCAexeDeviceReset(int *hConnect,* **char** **inData*, **int** *lloutData,* **char** **outData);*
hConnect (I) Handle returned by ARCAConnect.

inData (I)
{
“mode”: “forced”
}
Note: after forced reset it is necessary to perform a connection

lloutData (I) Specifies *outData* size. Advised 256

outData (O)
{
“reply”:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}

Description

This command simulates a power off/power on of the machine (after the reply).

You must reconnect when the reset is complete

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData


```
{
  "mode":"forced"
}
```


outData


```
{
  "reply":{
    "replyCode":"0",
    "replyCodeDescription":"OK"
  }
}
```

3.19 ARCAgetReaderCurrency

Result API ARCAgetReaderCurrency (**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"configType": "xxx"
}

Accepted value for configType:
"**bank**" : With this command it is possible to get the bank configuration on the reader

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content":
{

In case of bank
"BankN":{
"bankname":"xxxx",
"enabled":"Y/N",
"numbank":"xxxx"
},
"BankN":{
"bankname":"xxx",

```
                                "enabled":"Y/n",
                                "numbank":"xxxx"
                                }
                        }
}
```

Description

With this command it is possible to get the reader's bank configuration

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.20 ARCAgetReaderBanks

Result API ARCAgetReaderBanks(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
operationType : "xxxx"
}

Accepted value for **operationType**:

number :
With this command it is possible to get the number of active slots configured on the reader
configuration :
This command is used only with reader set in MULTICURRENCY mode and it is possible to get the bank configuration in use on the reader

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
{
"reply":
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content":
{
"ActiveBank":"xxx"
Or
"bankname" = "xxxx" ,

```
                    “enabled” = “xxxx”
                }
}
```

N.B in case of ActiveBank the reader response :
0 in case of Single currency mode
1- 4 in case of Multicurrency mode
> 4 =in case Single currency mode

Description

With this command it is possible to get the number of active slots configured on the reader

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.21 ARCAgetReaderVersion

Result API ARCAgetReaderVersion(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"versionType": "xxx"
}
Accepted value for **versionType**: ECB

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
```
{
    "reply":
    {
        "replyCode": "xxxx" ,
         "replyCodeDescription": "xxxx"
    }
    "content":
    {
         "ECB_header":
         {
              "FsID": "xxx",
              "name": "xxx",
              "ref_name": "xxx",
              "ref_version": "xxx",
              "rel": "xxx"
         }
     }
}
```

Description

With this command it is possible to get the reader version

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.22 ARCAgetCassetteSettings

Result API ARCAgetCassetteSettings (**int** *hConnect,*
string ** inData,*
int lloutData**,**
string **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"configType" : "xxxx" ,
"cassetteID" : "xxxx,
"bank" : "xxxx"
}

Accepted values for configType:

protocolCassetteNumber*= it is possible to get the number of the configured cassettes used by communication protocol*

serviceProtocolCassetteNumber=Number of configured cassettes used by service protocol

numCassette = With this command it is possible to get the number of the configured cassettes of the device

denominations = possible to know the denomination of the banknote
emissions that have been assigned to the multitype specific cassette (ddd#),
multi denomination specific cassette (dd*#) and multi nation specific cassette (####),
in this case you must specify the parameter *cassetteID*

references *:* used to read the complete set-up of the unit
in this case you must specify the parameter **bank**

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)

```
{
        "reply":
        {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
        }
        "content":
        {
                "numCassette" : "xxxx"
        }
}
```

Description

This command needs ARCAOpen to be issued before (apart from numCassette)

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata:

```
{
 "configType": "numCassette"
}
```

Outdata:

```
{
 "content": {
  "numCassette": "12"
 },
 "reply": {
  "replyCode": "0",
```

```
    "replyCodeDescription": "OK"
   }
  }
```


Indata:


```
  {
   "configType": "protocolCassetteNumber"
  }
```


Outdata:


```
  {
   "content": {
    "numCassette": "12"
   },
   "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
   }
  }
```


Indata:


```
  {
   "configType": "serviceProtocolCassetteNumber"
  }
```


Outdata:


```
  {
   "content": {
    "numCassette": "12"
   },
```

```
  "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
  }
}
```


Indata:


```
{
  "configType": "references",
  "bank": "0"
}
```


Outdata:


```
{
  "content": {
    "Denom_0": {
      "CassetteA": "N",
      "CassetteB": "N",
      "CassetteC": "Y",
      "CassetteD": "Y",
      "CassetteE": "Y",
      "CassetteF": "Y",
      "CassetteG": "N",
      "CassetteH": "N",
      "CassetteI": "N",
      "CassetteJ": "N",
      "CassetteK": "N",
      "CassetteL": "N",
      "Denom_Enable": "Y",
      "Denom_Height": "62",
```

```
  "Denom_Nid": "CPCA",
  "Denom_Version": "L000"
 },
 "Denom_1": {
  "CassetteA": "N",
  "CassetteB": "N",
  "CassetteC": "Y",
  "CassetteD": "Y",
  "CassetteE": "Y",
  "CassetteF": "N",
  "CassetteG": "Y",
  "CassetteH": "N",
  "CassetteI": "N",
  "CassetteJ": "N",
  "CassetteK": "N",
  "CassetteL": "N",
  "Denom_Enable": "Y",
  "Denom_Height": "67",
  "Denom_Nid": "CPDA",
  "Denom_Version": "L000"
 },
 "Denom_2": {
  "CassetteA": "N",
  "CassetteB": "N",
  "CassetteC": "Y",
  "CassetteD": "Y",
  "CassetteE": "Y",
  "CassetteF": "N",
```

"CassetteG": "N",
"CassetteH": "Y",
"CassetteI": "N",
"CassetteJ": "N",
"CassetteK": "N",
"CassetteL": "N",
"Denom_Enable": "Y",
"Denom_Height": "72",
"Denom_Nid": "CPEA",
"Denom_Version": "L000"
},
"Denom_3": {
"CassetteA": "N",
"CassetteB": "N",
"CassetteC": "Y",
"CassetteD": "Y",
"CassetteE": "Y",
"CassetteF": "N",
"CassetteG": "N",
"CassetteH": "N",
"CassetteI": "Y",
"CassetteJ": "N",
"CassetteK": "N",
"CassetteL": "N",
"Denom_Enable": "Y",
"Denom_Height": "77",
"Denom_Nid": "CPGA",
"Denom_Version": "L000"

},
"Denom_4": {
"CassetteA": "N",
"CassetteB": "N",
"CassetteC": "Y",
"CassetteD": "Y",
"CassetteE": "Y",
"CassetteF": "N",
"CassetteG": "N",
"CassetteH": "N",
"CassetteI": "N",
"CassetteJ": "Y",
"CassetteK": "N",
"CassetteL": "N",
"Denom_Enable": "Y",
"Denom_Height": "82",
"Denom_Nid": "CPHA",
"Denom_Version": "L000"
},
"Denom_5": {
"CassetteA": "N",
"CassetteB": "N",
"CassetteC": "Y",
"CassetteD": "Y",
"CassetteE": "Y",
"CassetteF": "N",
"CassetteG": "N",
"CassetteH": "N",

"CassetteI": "N",
"CassetteJ": "N",
"CassetteK": "Y",
"CassetteL": "N",
"Denom_Enable": "Y",
"Denom_Height": "82",
"Denom_Nid": "CPIA",
"Denom_Version": "L000"
},
"Denom_6": {
"CassetteA": "N",
"CassetteB": "N",
"CassetteC": "Y",
"CassetteD": "Y",
"CassetteE": "Y",
"CassetteF": "N",
"CassetteG": "N",
"CassetteH": "N",
"CassetteI": "N",
"CassetteJ": "N",
"CassetteK": "N",
"CassetteL": "Y",
"Denom_Enable": "Y",
"Denom_Height": "82",
"Denom_Nid": "CPJA",
"Denom_Version": "L000"
},
"ReaderConfig": {

```
    "Denomination_CassetteA": "BOXA",
    "Denomination_CassetteB": "BOXB",
    "Denomination_CassetteC": "CP--",
    "Denomination_CassetteD": "CP++",
    "Denomination_CassetteE": "CP??",
    "Denomination_CassetteF": "CPCA",
    "Denomination_CassetteG": "CPDA",
    "Denomination_CassetteH": "CPEA",
    "Denomination_CassetteI": "CPGA",
    "Denomination_CassetteJ": "CPHA",
    "Denomination_CassetteK": "CPIA",
    "Denomination_CassetteL": "CPJA",
    "Message": "0"
   }
  },
  "reply": {
   "replyCode": "0",
   "replyCodeDescription": "OK"
  }
 }
```

3.23 ARCAgetReaderStatistics

Result API ARCAgetReaderStatitics(**int** *hConnect,*
char ***inData**,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 4096

outData (O)

```
{
        reply:
        {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
        }
        "content":
        {
                "nAccept":
                {
                        "count": "xxx",
                        "description": "Number of accepted notes"
                },
                "nClsf":
                {
                        "count": "xxxx",
                        "description": "Number of notes that fail Classification"
                },
                "nDisableChain":
                {
                        "count": "xxx",
                        "description": "Number of notes refused due Disable Chain"
                },
                "nExtremeFit":
```

```
{
        "count": "xxxx",
        "description": "Number of notes that fail Extreme Fit"
 },
 "nFormat":
{
        "count": "xxxx",
        "description": "Number of notes that fail Format"
 },
 "nGenericError":
{
        "count": "xxxx",
        "description": "Number of notes that fail due generic error"
 },
 "nIrd":
{
        "count": "xxxx",
        "description": "Number of notes that fail Infrared"
 },
 "nMag":
{
        "count": "xxxx",
        "description": "Number of notes that fail Magnetic"
 },
 "nNotAuth":
{
        "count": "0",
        "description": "Number of notes considered not authenticated"
       },
 "nOcrCheck":
{
        "count": "xxxx",
        "description": "Number of notes that fail due OCR check"
       },
 "nOverrun":
{
        "count": "xxxx",
        "description": "Number of notes that fail Overrun"
       },
 "nProc":
{
        "count": "xxxx",
        "description": "Number of processed notes"
 },
 "nRefused":
{
        "count": "xxxx",
```

```
                        "description": "Number of refused notes"
                    },
                    "nSusp":
                    {
                        "count": "xxxx",
                        "description": "Number of notes considered suspect"
                    },
                    "nUnfit":
                    {
                        "count": "0",
                        "description": "Number of notes ECB Ok but UNFIT"
                    },
                    "nUv":
                    {
                        "count": "xxxx",
                        "description": "Number of notes that fail UV"
                    }
            }
}.
```


Description

With this command it is possible to get the reader rejects, sorted by types of reject action on the last deposit operation.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.24 ARCAsetUpperTransportLock

Result API ARCAsetUpperTransportLock (**int** *hConnect,* **char** **inData*, **int** *lloutData,* **char** **outData);*
hConnect (I) Handle returned by ARCAConnect.

inData (I)
{
“operation” : “xxx”
}
 	
Accetped values:
You can set “**operation**” = “lock” / “unlock” to manage the upperlock

lloutData (I) Specifies *outData* size. Advised 512

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

With this command it lock/unlock the upper area of Novus Machine (not available on CM18x)

Future enhancement

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.25 ARCAsetCassetteSettings

Result API ARCAsetCassetteSettings(**int** *hConnect,*
string ** inData,*
int lloutData**,**
string **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
“configtype“ : “xxxx” ,
“denomination” : “xxxx” ,
“denominationStatus” : “xxxx”
“cassetteID” : “xxxx,
“cassetteStatus” : “xxxx”
}

Accepted values for configType:

switch= Using this command it is possible to switch off/on a single cassette which is not working properly, in order to continue using the rest of the machine.
Needed parameter are :
cassetteID= target cassette (A-num_cas)
cassetteStatus= (enable/disable)

config : This command is used to assign an identifier-denomination to one or more cassette and/or enable/disable the denomination on the single cassette .
Needed parameter are :
denomination = denomination (4 ASCII-chars) (ex CPEA)
cassetteID= target cassette (A-num_cas) (ex D)
denominationStatus = (enable/disable)

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

The application can switch on/off only one cassette per time. After this command the application must perform the Open command before starting new cash in/out operations.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

indata


```
{
   “cassetteID” : "A",
   “cassetteStatus” : "disable",
   “configType” : "switch"
}
```


outData:


```
{
  "reply": {
    "replyCode": "0",
    "replyCodeDescription": "OK"
  }
}
```

3.26 ARCAsetConfigReader

Result API ARCAsetConfigReader (**int** *hConnect,*
string ** inData,*
int lloutData**,**
string **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
*"*denomination" : "xxxx" ,
"denominationStatus" : "xxxx"

}

Needed parameter are :
denomination = denomination (4 ASCII-chars) (ex EUEA)
denominationStatus = (enable/disable)

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
```
{
    reply:
    {
        "replyCode": "xxxx" ,
        "replyCodeDescription": "xxxx"
     }
}.
```

Description

With this command it is possible to enable/disable a given banknote on the reader

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.27 ARCAgetMacAddress

Result API ARCAgetMacAddress(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
        “reply”:
        {
                "replyCode": "xxxx" ,
                "replyCodeDescription": "xxxx"
        }
        "content":
       {
                “MacAddress” = “xxxx”
        }
}.
```

Description

With this command it is possible to know the MAC Address serial number

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Outdata


```
{
 "content": {
  "MacAddress": "000E728134E3"
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.28 ARCAgetReaderThreshold

Result API ARCAgetReaderThreshold(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
*"*configType" = "xxxx" ,
bank : "xxxx
}
"ConfigType" : possibile values:
"**fitConfiguration**" = it is possible to get the fitness configuration for all the references installed on the reader's flash memory.

"**fitThreshold**" = it is possible to get the threshold fitness in use for each bank

*"***fitVersion**" = "it is possible to know the threshold fitness version"

*"**bank**"* = Reference bank number (0-15)

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
{
"content" :
{" fitnessFlag "xxxx"}
Or
{" fitnessThreshold *=":"xxxx"}*
or
{" fitnessVer ":"xxxx"}

reply : {"replyCode":"xxxx","replyCodeDescription":"xxxx"}
}

fitnessFlag = 0= Fitness Enabled , 1= Always FIT
fitnessThreshold = Threshold Fitness in use ID (0:2)
fitnessVer = Version of threshold fitness

}.

Description

With this command it is possible to get some functionality parameters on the unit

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.29 ARCAgetDeviceSettings

Result API ARCAgetDeviceSettings(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 4096

outData (O)

```
{
   "content":
   {
      "Config":
      {
                  "alarm1Handling": "true/false",
                        "balancedCassetteHandling": " true/false ",
                        "delayClassDispensing": " true/false ",
                        "enableJournallog": " true/false ",
                        "useUnfitinSafe": " true/false ",
                        "viewDateTimeOnDisplay": " true/false ",
                        "viewDateTimeOnDisplayInFormatAMandPM": " true/false "
      },
      "OptionConfig":
      {
                        "executeCloseCommandAlsoInJam": " true/false ",
                        "simplifiedProtocol": " true/false ",
                        "specialCleanWithoutControl": " true/false "
       },
```

```
    "OptionOneConfig":
    {
                        "doNotRemoveCassetteJamIfSpecialCleanON": " true/false ",
                        "enableBookingExternalButton": " true/false ",
                        "forceRecoveryWithTestCommand": " true/false ",
                        "lanTimeout": " true/false ",
                        "timeControlDispenseamount": " true/false "
    },
   "OptionTwoConfig":
   {
                        "alarm2OutputHandling": " true/false ",
                        "bvuSynchronousHandling": " true/false ",
                        "cat2BoxHandling": " true/false ",
                        "differentCenterAUDbn": " true/false ",
                        "disableCM18evoLedPulse": " true/false ",
                        "disableMessageonDisplay": " true/false ",
                        "evoLedHandling": " true/false ",
                        "faceOrientationHandling": " true/false ",
                        "feedBanknoteswithInputBinFree": " true/false ",
                        "handlingNewLedSide": " true/false ",
                        "remapFeederJamtoMissfeeding": " true/false ",
                        "safeCen40Bit": " true/false ",
                        "separateCat2andCat3": " true/false ",
                        "specialCleanUsingDisplay": " true/false ",
                        "virtualDestkopEnvironment": " true/false "
    }
  },
  "reply":
  {
        "replyCode": "xxx",
        "replyCodeDescription": "xxx"
  }
}.
```

Description

With this command it is possible to know the parameters set on the CM

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

outdata


```
{
 "content": {
  "Config": {
   "alarm1Handling": "true",
   "balancedCassetteHandling": "true",
   "delayClassDispensing": "false",
   "enableJournallog": "false",
   "useUnfitinSafe": "true",
   "viewDateTimeOnDisplay": "true",
   "viewDateTimeOnDisplayInFormatAMandPM": "false"
  },
  "OptionConfig": {
   "executeCloseCommandAlsoInJam": "false",
   "simplifiedProtocol": "true",
   "specialAlarmMode": "true",
   "specialCleanWithoutControl": "true"
  },
  "OptionOneConfig": {
   "donotremovecassettejamifspecialcleanON": "true",
   "enableBookingExternalButton": "false",
   "forceRecoveryWithTestCommand": "true",
   "lanTimeout": "false",
   "timeControlDispenseamount": "false"
  },
  "OptionTwoConfig": {
   "RtcModuleNameCompatibility": "false",
```

```
    "alarm2OutputHandling": "false",
    "bvuSynchronousHandling": "true",
    "cat2BoxHandling": "false",
    "differentCenterAUDbn": "false",
    "disableCM18evoLedPulse": "false",
    "disableMessageonDisplay": "false",
    "evoLedHandling": "false",
    "faceOrientationHandling": "false",
    "feedBanknoteswithInputBinFree": "false",
    "handlingNewLedSide": "false",
    "remapFeederJamtoMissfeeding": "false",
    "safeCen40Bit": "false",
    "separateCat2andCat3": "true",
    "specialCleanUsingDisplay": "false",
    "virtualDestkopEnvironment": "false"
   }
  },
  "reply": {
   "replyCode": "0",
   "replyCodeDescription": "OK"
  }
 }
```

3.30 ARCAsetAlarm

Result API ARCAsetAlarm (**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"alarmID"= "xxx"
"alarmStatus" ="enable/disable"
}

Accepted values for **alarmID** = alarm1 , alarm2 , alarmSilent

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

With this command it is possible to manage ALARM1/ALARM2. If you want to use this command the user doesn't want to manage TEMPORARY OPEN DOOR and ANTISISMIC ALARM

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.31 ARCAgetDelays

Result API ARCAgetDelays (**int** *hConnect,*
string ** inData,*
int lloutData**,**
string **outData)*;

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)

```
{
   "content": [{
                        "amount": "xxx",
                        "delay": "xxx",
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
                        "delay": "xxx"
                }, {
                        "amount": "xxx",
```

```
                    "delay": "xxx"
                }, {
                    "amount": "xxx",
                    "delay": "xxx"
                }, {
                    "amount": "xxx",
                    "delay": "xxx"
    }],


    “reply”:
    {
        "replyCode": "xxxx" ,
        "replyCodeDescription": "xxxx"
     }
}.
```

Description

With this command it is possible to get delay class.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Outdata:


```
{
 "content": [
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
   "delay": "0"
  },
  {
   "amount": "0",
```

```
    "delay": "0"
   },
   {
    "amount": "0",
    "delay": "0"
   },
   {
    "amount": "0",
    "delay": "0"
   },
   {
    "amount": "0",
    "delay": "0"
   },
   {
    "amount": "0",
    "delay": "0"
   }
  ],
  "reply": {
   "replyCode": "0",
   "replyCodeDescription": "OK"
  }
}
```

3.32 ARCAgetAudioSettings

Result API ARCAgetAudioSettings(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
operationType : "getAudioValue"
}

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
"reply":
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content":
{
*"*audioValue" : "xxxx"
}
}

Description

With this command it is possible to get the audio configuration parameters

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.33 ARCAexeAlarm

Result API ARCAexeAlarm(**int** *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"type" : " xxx" ,
"lockMode" : "xxx" ,
"password" : "xxx" ,
"lockTime"= "xxx"
}

Accepted values:
type = passwordLock , temporaryLock

in case of **passwordLock**

lockMode value accepted = lockOn , lockOff
password = Keycode Access (6 ASCII-chars)

in case of **temporaryLock** :
lockTime = <= 5 lock time = 5 minutes
>=30 lock time = 30 minutes
5 < lockTime < 30 lock time = lockTime minutes

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,

```
        "replyCodeDescription": "xxxx"
    }
}.
```

Description

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.34 ARCAgetReaderIdentifier

Result API ARCAgetReaderIdentifier(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
"type" : " xxx"
}

Acceted values:
type = version, references

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)
```
{
    "content":
    {
      "cdfName":"xxxx"
    },
    "reply":
    {
      "replyCode":"xxxx",
      "replyCodeDescription":"xxxx"
    }
                Or
    "content":
    {
      "References":
        {
          "0":{"bankId":"xxx","ref_name":"xxxx","ver":"xxxx"},
          "xxx":{"bankId":"xxx","ref_name":"xxxx }
```

```
        },
    }
    "reply":
    {
       "replyCode":"xxxx",
       "replyCodeDescription":"xxxx"
    }
}
```

Description

Returns the CDF version or the reference ID with the related bank information

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

Indata

```
{
 "type": "version"
}
```

outdata

```
{
 "content": {
  "cdfName": "RNH_02_036_11"
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
```

```
}
```

indata

```
{
 "type": "references"
}
```

outdata

```
{
 "content": {
  "References": {
   "0": {
    "bankId": "0",
    "ref_name": "EU",
    "ver": "G010"
   },
   "1": {
    "bankId": "1",
    "ref_name": "CP",
    "ver": "L000"
   }
  }
 },
 "reply": {
  "replyCode": "0",
  "replyCodeDescription": "OK"
 }
}
```

3.35 ARCAInitAsyncEvent

Result API ARCAInitAsyncEvent(int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
“enableEvent” : “ xxx”,
“disableEvent” : “ xxx”,
“errorsEvent” : “ xxx”,
“safeEvent” : “ xxx”,
“pocketsEvent” : “ xxx”
}

“possible values : true/ false

"disableEvent" = “all Events disabled”
"enableEvent" = “= all Events enabled”
" errorsEvent " = “all Errors”
"pocketsEvent" = “all bins events”
“SafeEvent" = ”Safe door, Upper transport, Cage event”

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply:
{
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
}.

Description

With this command it is possible to enable or disable the different events (possible only with Novus)

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

indata

```
{
  disableEvent : "false"
  enableEvent : "true"
  errors : "false"
  pocketsEvent : "false"
  safeEvent : "false"
}
```

3.36 ARCAsetDelays

Result API ARCAsetDelays (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

```
{
        “content”: [
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                        { “amount”: “xxx”, “delay”: “xxxx” },
                ]
}
```

- amount= The amount is in about a hundred unit (example: if amount = 1 the real value is 1 * 100 = 100)
- delay= Delay in seconds for this class (0-65535)

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
    reply:
    {
        "replyCode": "xxxx" ,
        "replyCodeDescription": "xxxx"
     }
}.
```

Description

This command is used to fill the delay class to the machine stored in controller

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

3.37 ARCAexeTraceTransfer

Result API ARCAexeTraceTransfer (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

JSON input data structure:

```
{
    "transferType" : "xxxx" ,
    "transferMode" : "xxxx" ,
    "transferFile" : "xxxx"
     "ZipfileName" : "xxxx"
}
```

Accepted values for **transferType**: :
Download
Accepted values for **transferMode**:
Service
Accepted values for **transferFile**
Log

In case of Log you can specify "ZipfileName" (Name zip saved)

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)

```
{
   reply:
   {
      "replyCode": "xxxx" ,
      "replyCodeDescription": "xxxx"
    }
}.
```

Description

This command is used to get all LOG and service data and save them in a zip File

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  TransferFile : "Log"
  TransferMode : "Service"
  TransferType : "Download"
  ZipfileName : "C: \\test.zip"
}
```

outData

```
{
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.38 ARCAexeCassetteInit

Result API ARCAexeCassetteInit (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:

```
{
    "TargetToinitialize " : "xxxx" ,
    "code " : "xxxx"
}
```

Accepted values for :
TargetToinitialize : Denomination to initialize (Es: EUCA) or Cassette to initialize (Es: AAAA)
code : Cassette key-code

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)

```
{
    "content":
    {
      "message": xxx
     }
    "reply": { "replyCode": "xxxxx", "replyCodeDescription": "xxxx" } }
```

Return value :
message*:* **0** = Ok, **1,3,5** = reserved, **2**= cassette not present
4= wrong code, **6**= denomination not empty

}.

Description

When the 'Note counter' of a cassette does not match the physical status of the same, after the complete withdrawal, the init cassette command has to be used in order to avoid the mismatching.(If some notes are manually removed after a cassette jam, the init command can be used to reset the cassette and initialize the logical note counter). For both user_name CASH BALANCE OPERATOR is set to zero if this command is correctly executed (reply code = OK and message = 0).

This command request ARCAOpen to be issued before

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData {

TargetToinitialize : "DDDD"

code : "123456"

}

outData

{

content : {"message":0}

reply : {"replyCode":"0","replyCodeDescription":"OK"}

}

3.39 ARCAgetPocketOutput

Result API ARCAgetPocketOutput (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:
{
*"*configType*" : "xxxx" ,*
}

Accepted values for **configType***:*
bundleSize-> get the maximum number of banknotes in output slot during the operation of withdrawal

lloutData (I)
Specifies *outData* size.
Advised 512

outData (O)
{
JSON output data structure:

*In case of "*bundleSize"{

```
reply:
{
    "replyCode": "xxxx" ,
    "replyCodeDescription": "xxxx"
}
"content":
{
     "bundleSize": "xxx"
}
```

}.

Description

With this command it is possible to get the number of banknotes on output slot during the operation of withdrawal.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  noteType : "bundleSize"
}
```

outData

```
{
  content : {"bundleSize":"200"}
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.40 ARCAsetPocketOutput

Result API ARCAsetPocketOutput (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:
JSON input data structure:
{
*"*configType*" : "xxxx" ,*
*"*numnotes" : "xxx"

}
Accepted values for **configType***:*
withdrawalBundleSize -> set the maximum number of banknotes in output slot during the operation of withdrawal

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}

}.

Description

With this command it is possible to set the number of banknote on output slot during the operation of withdrawal.

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  configType : "withdrawalBundleSize"
  numnotes : "200"
}
```

outData

```
{
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.41 ARCAgetPocketInput

Result API ARCAgetPocketInput (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:

{
"configType" : "xxxx" ,
}

Accepted values for **configType***:*
"bundleSize

lloutData (I)
Specifies *outData* size.
Advised 512

outData (O)
{
JSON output data structure:

*In case of "*bundleSize"
{
reply: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content": {
"bundleSize": "xxx"
}

}.

Description

With this command it is possible to get the number banknote on output slot during the operation of counting

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  configType : "bundleSize"
}
```

outData

```
{
  content : {"bundleSize":"200"}
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.42 ARCAsetPocketInput

Result API ARCAsetPocketInput (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:
JSON input data structure:
{
*"*configType*" : "*bundleSize*" ,*
*"*numnotes" : "xxx"
}

lloutData (I)
Specifies *outData* size.
Advised 256

outData (O)
{
reply: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}

}.

Description

With this command it is possible to set the number of banknotes in output slot during a counting operation (the default is 200)

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  configType : "bundleSize"
  numnotes : "200"
}
```

outData

```
{
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.43 ARCAsetLogServiceRecord

Result API ARCAsetLogServiceRecord (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:

JSON input data structure:

```
{
        “nn1” : “xxxx” ,
        “nn2” : “xxxx” ,
        “nn3” : “xxxx” ,
        “nn4” : “xxxx” ,
        “nn5” : “xxxx” ,
        “nn6” : “xxxx” ,
        “nn7” : “xxxx” ,
        “nn8” : “xxxx” ,
        “nn9” : “xxxx” ,
        “nna” : “xxxx” ,
        “nnb” : “xxxx” ,
        “nnc” : “xxxx” ,
        “nnd” : “xxxx” ,
        “nne” : “xxxx” ,
        “nnf” : “xxxx” ,
        “nng” : “xxxx” ,
        “nnh” : “xxxx” ,
        “nni” : “xxxx” ,
        “nnj” : “xxxx” ,
        “nnk” : “xxxx”
}
```

- Nn1-nnk = Hexadecimal data. (two Hex Ascii)
- CM18 /CM18E: nn1-nnb
- CM20E: nn1-nne
- CM18T: nn1-nnk

lloutData (I) Specifies *outData* size. Advised 256

outData (O)
{
reply: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
 	
}.

Description

With this command it is possible to set a log record . **FUTURE ENHANCEMENT**

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

3.44 ARCAgetModuleSerialNumber

Result API ARCAgetModuleSerialNumber (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

JSON input data structure:
{
" moduleName" = "RTC"
}

lloutData (I)
Specifies *outData* size.
Advised 512

outData (O)
{
reply: {
"replyCode": "xxxx" ,
"replyCodeDescription": "xxxx"
}
"content": {
"serialNumber": "xxx"
}
}.

Description

With this command it is possible to know the controller serial number(RTC)

Return Value

AR_OKAY if successful

Refer to status.json for the full list and description of return codes.

Example

inData

```
{
  moduleName : "RTC"
}
```

outData

```
{
  content : {"serialNumber":"R123451234567891"}
  reply : {"replyCode":"0","replyCodeDescription":"OK"}
}
```

3.45 ARCAgetCoinCashData

Result API ARCAgetCoinCashData (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)

{
*"*operationType *" : "xxxx"*
}

operationType parameter : **dispensedAmount**, **availableAmount**

lloutData (I)
Specifies *outData* size.
Advised 1024

outData (O)
{
{
content : {"availableAmount":"xxx"}
reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}
}
Or
{
content : {"dispensedAmount":""}
reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}
}
}.

Description

This command request ARCAOpen to be issued before

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes.

3.46 ARCACoinInventory

Result API ARCACoinInventory (int *hConnect,* **char** **inData*, **int** *lloutData,* **char** **outData);*
hConnect (I) Handle returned by ARCAConnect.
inData (I) *{* *"operationType " : "xxxx"* *"amount" : "xxx"* *}* operationType: accepted parameter : **load**, **clear**
lloutData (I) Specifies *outData* size. Advised 256
outData (O) { *reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}* }.

Description

clear : With this command it is possible to reset the total amount of the Coin Dispenser stored on the device. The same command resets the counters of the amount exempted for each side (or operator).

Load : With this command it is possible to store the amount loaded in the Coin Dispenser in the device.

amount : "xxx" = Amount deposited. The sum should be specified without a decimal point. ".". The last two digits will be treated as decimals. Example: 123 means 1 dollar and 23 cents; 12,300 means $ 123.

This command request ARCAOpen to be issued before

Parameters

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

3.47 ARCACoinDispense

Result API ARCACoinDispense (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
*"*operationType *" : "xxxx"*

amount : "xxx"
or
{
coinToDispense :
[{"denominationID":"xxx","value":xxx"}
,{"denominationID":"xxx","value":"xxxx ,
…..

denominationID":"xxx","value":"xxxx"}]
}
}

operationTypeaccetpted parameter **: *amount*, *denomination***

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)
{
reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}
}.

Description

***amount*:** With this command it is possible to obtain dispense by amount

denomination = Every de nomination that belong to **den_list** is composed from a couple of numbers. Therefore, it can take values from **00** to **99**. If the command contains only a name consisting of more than two digits, this is not executed. The same applies if one or more names are expressed with less than 2 digits.

Each name is separated by a comma ",". **The names are sorted in descending order**

T**his command request ARCAOpen to be issued before**

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

3.48 ARCAgetCoinConfiguration

Result API ARCAgetCoinConfiguration (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
{
*"*coinParamId *" : "xxxx"*

}

coinParamId :
"0": Coin denomination for hopper 0
…….
"7": Coin denomination for hopper 7
"d": Value of the notes below (in cents)
"a": List of all parameters + dispense below + type canister
"t": Type canister (USA, Canada, etc…)

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)

In case 0 to 7 or d
{
content : {"coinParamValue":"xxx"}
reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}
}

In case of a
{
content :
{"den0":"xxx",

```
"den1":"xxx"
 "den2""xxx",
 "den3":"xxx",
 "den4":"xxx",
 "den5":"xxx",
 "den6":"xxx"
 ,"den7  ":"xxx"}
 reply : {"replyCode":"xxx","replyCodeDescription":"xxx"}
 }
```

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

3.49 ARCAgetCassettePhysicalPosition

Result API ARCAgetCassettePhysicalPosition (int *hConnect,*
char **inData*,
int *lloutData,*
char **outData);*

hConnect (I)
Handle returned by ARCAConnect.

inData (I)
Must be a no NULL pointer, no input json data

lloutData (I)
Specifies *outData* size.
Advised 2048

outData (O)

```
        {
    "content": {
        "cassetteA": {
            "position": "xx"
        },
        "cassetteB": {
            "position": "xx"
        },


        "cassettexx": {
            "position": "xx"
        },
       "numCassette":"xx"
    },
    "reply": {
        "replyCode": "xx",
        "replyCodeDescription": “xx"
    }
}
```

Description

This device introduced the feature of **Swappable Cassette**. It means that cashiers and technicians can insert cassettes inside the device without the needs of respecting the order of cassettes. Command like CashData, replies to withdrawal and deposits will return cassettes content using the logical naming of cassettes

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

Example

outData


```
{
    "content": {
        "cassetteA": {
            "position": "9"
        },
        "cassetteB": {
            "position": "10"
        },
        "cassetteC": {
            "position": "3"
        },
        "cassetteD": {
            "position": "4"
        },
        "cassetteE": {
```

```
                "position": "1"
            },
            "cassetteF": {
                "position": "2"
            },
            "cassetteG": {
                "position": "7"
            },
            "cassetteH": {
                "position": "8"
            },
            "cassetteI": {
                "position": "5"
            },
            "cassetteJ": {
                "position": "6"
            },
            "cassetteK": {
                "position": "11"
            },
            "cassetteL": {
                "position": "12"
            },
            "numCassette":"12"
        },
        "reply": { "replyCode": "201",    "replyCodeDescription": "not found"}
}
```

3.50 ARCAgetBanknoteSerialNumber

Result API ARCAgetBanknoteSerialNumber int *hConnect,* **char** **inData,* **int** *lloutData,* **char** **outData);*
hConnect (I) Handle returned by ARCAConnect.
inData (I) *{* *numberBkn : "xxx"* *}*
lloutData (I) Specifies *outData* size. Advised 512

outData (O)
 	
{
"content ": {"numberBkn":"XXX","*denomBkn" : "XXXX",*
"serialNumber":"XXXX","typeCat":"XXX","typeOcr":"XXX"}
reply : {"replyCode":"XXX","replyCodeDescription":"XXX"}
}

Description

This function is used to return the S/N of the banknote deposited/dispensed/counted (**only for Novus**)

numberBkn start from 1 , the application must call this function in a loop until the reply Code is 219

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

~~**Example**~~

~~inData~~

~~{~~

~~"numberBkn" : "4"~~

~~}~~

~~outData~~

~~{~~

~~"content" : {"numberBkn":"1",~~
~~"denomBkn":"USAA",""serialNumber":"J58957484C","typeCat":"1","typeOcr":"0"}~~

~~"reply" : {"replyCode":"0","replyCodeDescription":"OK"}~~

~~}~~

3.51 ARCAexeDeviceBell

Result API ARCAexeDeviceBell int *hConnect,* **char** **inData*, **int** *lloutData,* **char** **outData);*
hConnect (I) Handle returned by ARCAConnect.
inData (I) *{* numBeep*: "xxx"* *}*
lloutData (I) Specifies *outData* size. Advised 256

outData (O)
{
reply : {"replyCode":"XXX","replyCodeDescription":"XXX"}
}

Description

With this command is possible to execute some acoustic signal

Return Value

AR_OKAY

AR_UNIT_ALREADY_RESERVED

AR_UNIT_BUSY

Refer to status.json for the full list and description of return codes

Example

inData

{

numBeep : "4"

}

outData

{

reply : {"replyCode":"0","replyCodeDescription":"OK"}

}